5. Sampling of continuous-time

5.1 Introduction

The conversion of continuous-time signals to digital signals (analog-to-digital conversion) consists of three steps:

- sampling, i.e. converting a continuous-time signal to a discrete-time signal (discrete signal)
- quantization, i.e. expressing the values of discrete samples via a finite set of numbers (numbers of finite length), with quantization noise being produced in the process
- coding, i.e. converting quantized samples to binary numbers in a certain code (inverse code, complement code, etc.).

In both practice and literature, the digital signal processing is referred to as a general notion where both the discrete signal and the digital signal are digitally processed. To show the difference between a discrete and a digital signal we will give some fundamental notions from the area of quantizing and coding discrete signal values.

We understand the discrete signal to be a sequence of numbers, the values of which can be expressed by a theoretically infinite number of digits in the given numeric system. We frequently want to implement algorithms for discrete signal processing such as frequency filtering or calculation of the spectrum of the discrete Fourier transform on a technical facility. The technical facility can be represented either by a digital computer, e.g. a PC, or by a single-chip digital signal processor or also by some special single-purpose hardware. In most cases the numerical values are saved in the binary form in memory registers of finite word lengths. For example, we work with input data whose individual values are expressed by binary numbers of b bits in length. If we multiply one b-bit sample by a b-bit coefficient, the result is $2b$ bits long. But memory registers or buses are only b bits long so that the result of multiplication must be reduced to b bits in some way. The process of replacing the resultant exact value by a shortened value is referred to as **quantization**. Various numerical systems can be used to express the numerical values inside a technical facility, the values of coefficients in memories or samples on the output of analog-to-digital (A/D) converter. The most widely used among them are polyadic numerical systems. In computer and microprocessor technologies, practically all applications are based on the binary (dyadic) numeral system. In the binary system, every non-negative number N is expressed as a sum of powers of numbers with a radix of 2 in the form

$$N = \sum_{i=-\infty}^{m} a_i 2^i, \quad a_i = 0 \text{ or } 1 \tag{5.1}$$

For example, the binary number $(11.101)_2 = (1 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3})_{10} = (3.625)_{10}$, can in the decimal numeral system be expressed as the positive number 3.625. In the following, the notation $(\,)_2$ will mean a number expressed in the binary numeral system, and the symbol $(\,)_{10}$ will mean a number in the decimal numeral system. The non-negative integer part (the subscript i of digits a_i acquires the values $i \geq 0$) and the fractional part of the binary number (the subscript i of digits a_i acquires the values $i < 0$) are separated by the decimal or binary points. Depending on whether the decimal point is fixed in one position or changes its position, we divide systems into **fixed-point** and **floating-point systems**. To enable binary expression of also negative numbers, the first bit of the numeric expression is usually taken as the **sign bit**. For instance, the number –3.625 can be expressed as $(111.101)_2$. When designing computers, it turned out to be of advantage from the viewpoint of the quantizing operation to use only the fraction part of the number. Using only the fraction part of the binary number, the quantizing operation is simple in that the bits with the lowest weight are just separated - truncated - or prior to being separated they are appropriately modified – rounded.

In the memory register of a technical device such as the microprocessor the number is stored as an integer but in most cases it is assumed that the first bit gives the sign of the number followed by the imaginary fixed decimal

point. The remaining part of the number is referred to as the **fraction part**. The latter is sometimes erroneously called mantissa but this term refers to this part only in floating-point numbers. A given microprocessor or computer always has a defined form of expressing the different types of variables (real, integer, double precision, etc.). The first digit of the fractional part with the highest weight is denoted with the symbol MSB (Most Significant Bit) and the last valid digit with the lowest weight has been assigned the symbol LSB (Least Significant Bit). Thus, for example, in the 24-bit word of the fixed-point digital signal processor of the DSP56300 (Motorola) family the first bit gives the sign, the second bit gives MSB and the twenty-fourth bit is LSB. The fraction part is 23 bits long. By means of the sign bit the binary numbers can be divided into positive and negative numbers. This way of expressing binary numbers with fixed decimal point is called sign and fraction part representation. Naturally, it is tacitly assumed that the fraction part always has a non-negative value. An example of representing positive and negative numbers in this form of expressing can be as follows

$$(0.101)_2 = (0.625)_{10} \quad \text{and} \quad (1.101)_2 = (-0.625)_{10}$$

With this representation, we work in summation with addends as positive binary numbers and the sign bits are evaluated by a special logic of the arithmetic unit, and depending on the type of operation and on the signs of the two addends the sign of the result is determined. The necessity of detecting the value of the sign bit in the summation and subtraction operations brings technical complications and therefore other possibilities were sought how to express negative numbers without the need to distinguish whether the operation is summation or subtraction. One option is to represent negative numbers using the inverse code (also called one's complement, notation $(\ )_{1'd}$). Another possibility is to use the representation that goes under the name **complement code** (two's complement, notation $(\ )_{2'd}$). The subtraction operation is performed in the same way as the summation of two positive numbers because the negative sign is encoded in the representation of the binary number. The conversion of a negative number to a positive number in the inverse code is done by simply inverting all the digits of the positive number as follows

$$(0.625)_{10} = (0.101)_{1'd} \text{ and } (-0.625)_{10} = (1.010)_{1'd}$$

The reason why this code is called inverse code is that it in fact represents the complement of the binary representation of a positive number with respect to a number which has logical ones in all valid places (e.g. for a 4-bit number it is the complement to the number 1.111). The term complement code has been introduced because the representation of a negative number is obtained by subtracting the positive binary number from the value $(2)_{10} = (10)_2$. In the microprocessor arithmetic this operation is performed in such a way that all the bits are inverted as in the inverse code, and the logical one is added to the least significant bit. An example of converting a positive number to a negative one in the complement code is as follows

$$(0.625)_{10} = (0.101)_{2'd} \quad \text{and} \quad (-0.625)_{10} = (1.011)_{2'd}$$

In Table 5.1 we see the representation of a binary 4-bit number in all the three formats. In the sign-magnitude part format and in the inverse code format the same value 0 is expressed by two codes (±0). The complement code actually has the advantage that if the range of representation is exceeded in the summation (or subtraction), no error occurs if in the subsequent summation or subtraction the result again falls into the correct range. This advantageous property in the case of overflow in the complement code can be made clear if we plot the values from Table 5.1 onto a circle as shown in Fig. 5.1, for example, for $b = 3$ bits (i.e. the binary number length is $b + 1 = 4$ bits). Because of these advantageous properties, the complement code is used to represent numbers in arithmetic units and hardware multipliers of microprocessor circuits.

Table 5.1 Comparison of three fixed-point representations of numbers

Binary number	Sign + fraction part	Inverse code	Complement code
0111	7/8	7/8	7/8
0110	6/8	6/8	6/8
0101	5/8	5/8	5/8
0100	4/8	4/8	4/8
0011	3/8	3/8	3/8
0010	2/8	2/8	2/8
0001	1/8	1/8	1/8
0000	+0	+0	+0
1000	-0	-7/8	-1
1001	-1/8	-6/8	-7/8
1010	-2/8	-5/8	-6/8
1011	-3/8	-4/8	-5/8
1100	-4/8	-3/8	-4/8
1101	-5/8	-2/8	-3/8
1110	-6/8	-1/8	-2/8
1111	-7/8	-0	-1/8

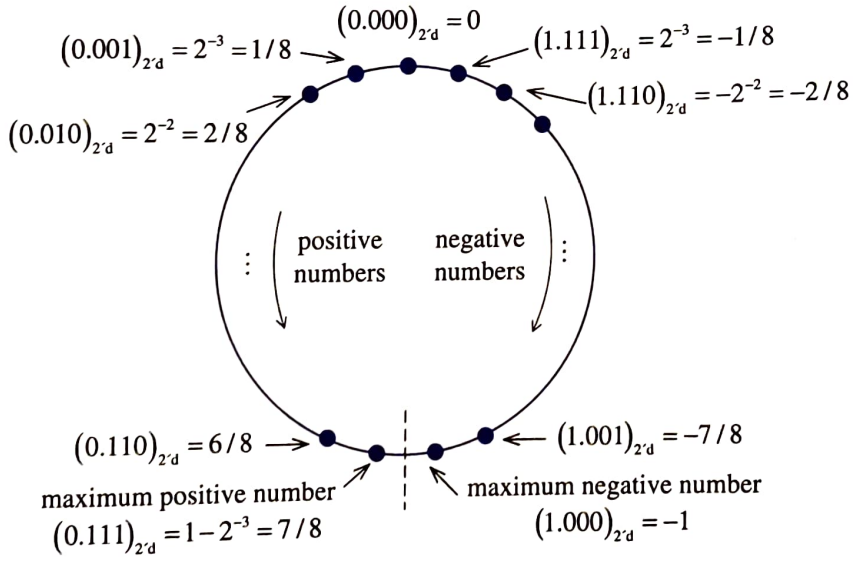


Figure 5.1: Ordering of values in complement code for 4-bit word length [RAB-75]

Let us illustrate the property of the complement code on an example

Decimal system		Binary system
0.84375	⇒	0.11011
+ 0.50000	⇒	+ 0.10000
1.34375 (overflow)	⇒	1.01011 ⇒ $(-0.65625)_{10}$
- 0.56250	⇒	+ 1.01110
0.78125	⇒	(1) 0.11001 ⇒ $(0.78125)_{10}$

Any digit that exceeds the memory register length to one side or the other is lost. That is why the transfer of one to the order of 2^1 is neglected. The decimal equivalent of a negative b-bit number in the complement code has the form

$$-\left[2-\left(1+\sum_{i=1}^{b} a_i 2^{-i}\right)\right] = \sum_{i=1}^{b} a_i 2^{-i} - 1 \tag{5.2}$$

The operations used in quantizing are **truncation** and **rounding**. For the sake of clarity, let us multiply two numbers, each of $b+1=4$ bits in length, given in the sign-fraction part representation

$$(0.101)_2 = (0.625)_{10} \quad \text{and} \quad (1.011)_2 = (-0.375)_{10}$$

The product of the numbers is

$$N = (1.001111)_2 = (-0.234375)_{10}$$

As can be seen, the length of the result is $2b+1=7$ bits. Now it is necessary to reduce the result to the length $b+1=4$ bits. In Fig. 5.2 the situation is indicated for the two ways of reducing. The quantity $Q = 2^{-b}$ is called the quantization step.

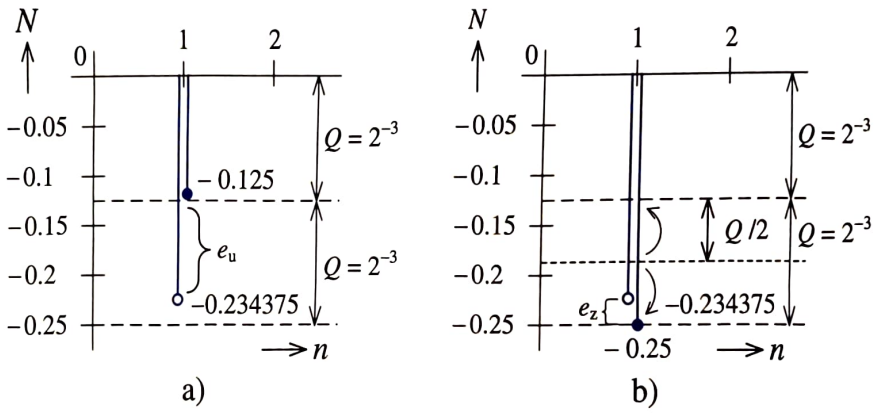


Figure 5.2: Ways how to reduce numbers: a) quantization, b) rounding

The truncation operation will be performed by simply removing the least significant bits from the result. In our case, the product after truncation has the form $(1.001)_2 = (-0.125)_{10}$. The truncation error is given by the difference between the reduced and the exact values

$$e_u = N_Q - N = -0.125 - (-0.234375) = (0.109375)_{10}$$

The symbol N_Q denotes the post-quantization value of the number.
The rounding operation will proceed so that first the non-reduced result will be increased by half the quantization interval, i.e. the value $Q/2$, and then the result will be truncated to $b+1$ bits. In our case the procedure is as follows

$$\begin{array}{ll} 1.001111 & N \\ +\,0.000100 & Q/2 \\ \hline 1.010011 & (1.010)_Q = (-0.25)_{10} \end{array}$$

The symbol $(\,)_Q$ denotes the quantization operation. In a similar way we also perform rounding in the decimal system. We want to round the numbers 11.4 and 11.6 to units. One half of the quantization step equals $Q = 10^0/2 = 0.5$.

In the first case, $11.4 + 0.5 = 11.9$ and after truncation the result is 11. In the second case, $11.6 + 0.5 = 12.1$ and after truncation the number is 12. The error of the rounded product is

$$e_z = N_Q - N = -0.25 - (-0.234375) = (-0.015625)_{10}$$

Comparing the two operations, we see that truncation is a simpler operation but the error can be greater (as much as one quantization step). Rounding exhibits a smaller error (approximately only up to half the quantization step) but the logical operations necessary to perform the operation are more complex.

A **binary floating-point number** can be represented in the following way

$$N = 2^c m$$

where m is the **mantissa** and c is the **exponent**. For example, the number $N_1 = 3$ can be represented as $N_1 = (4 \cdot 0.75)_{10} = (2^{10} \cdot 0.110)_2$. How the mantissa and the exponent are represented for negative numbers depends on the proposed arithmetic unit of the technical facility. In most cases, the two parts are usually represented in the complement code. For example, for floating-point microprocessors it is recommended to use the IEEE Standard 754 for arithmetic units. The floating-point numbers are then represented in the same format and various types of microprocessor can cooperate when transferring the results. To obtain the maximum possible precision and to make use of the dynamic range of the mantissa, the latter is often normalized, i.e. a check is made of whether the mantissa value is within the range

$$0.5 \le m < 1 \tag{5.3}$$

As an example, let us multiply the number $N_1 = 4 \cdot 0.75 = (2^{10} \cdot 0.110)_2$ by the number $N_2 = (4 \cdot 0.625)_{10} = (2^{10} \cdot 0.101)_2$

$$N_1 \cdot N_2 = (2^{10} \cdot 0.110)(2^{10} \cdot 0.101) = 2^{(10+10)}\,(0.110 \cdot 0.101) = 2^{100} \cdot 0.011110$$

When normalizing without considering subsequent quantization, the result of multiplication has the form

$$N_1 \cdot N_2 = (2^{011} \cdot 0.111100)_2$$

In a processor, the floating-point number is again represented as an integer with imaginary placement of decimal points for the mantissa and the exponent. In microprocessors the exponent length is usually ca ¼ of the overall length. For example, in the Motorola DSP96002 digital signal processor the exponent is 8 bits long, the mantissa with sign is 24 bits long and the word length is thus 32 bits.

The dynamic range of a given representation $\Re$ is usually defined as the proportion of the maximum positive number to the minimum positive number, in terms of logarithmic scale (in decibels-dB). For fixed-point systems, all the three number representations considered (sign-magnitude part, inverse code, and complement code) have the same values for positive numbers. The maximum positive number in the fraction representation is the number $N_{max} = 1 - 2^{-b}$, the minimum positive number is $N_{min} = 2^{-b}$. The representation range is

$$\Re_{INT} = 20\log\frac{N_{max}}{N_{min}} = 20\log\frac{1-2^{-b}}{2^{-b}} = 20\log(2^b - 1) \sim 6.0206 \cdot b \tag{5.4}$$

The index INT (INTeger) denotes a system with fixed decimal point. For floating-point systems let the exponent be b_e bits long and the mantissa b_m bits. The maximum positive number that can be represented is $N_{max} = 2^{(2^{b_e}-1)} \cdot (1 - 2^{-b_m}) \sim 2^{(2^{b_e}-1)}$ while the minimum positive number is $N_{min} = 2^{-2^{b_e}}$. The dynamic representation range is

$$\Re_{FLP} = 20\log\frac{N_{max}}{N_{min}} = 20\log 2^{(2^{be}-1+2^{be})} \sim 6.0206 \cdot (2^{be+1}-1) \quad (5.5)$$

... FLP (Floating Point) will refer to floating-point systems. Fig. 5.3 gives a comparison of the dynamic range ... numbers of bits. The fraction part b of the system of type INT is compared in a simplified manner with the exponent b_e of the system of type FLP. The corresponding values can be seen in Table 2.

Table 5.2 Comparison of dynamic ranges of type INT and type FLP systems (fraction part is compared with exponent length)

number of bits	1	2	3	4	5	6	7	8
INT [dB]	6.0206	12.041	18.062	24.082	30.103	36.124	42.144	48.165
FLP [dB]	18.062	42.144	90.103	189.64	379.30	764.62	1535	3076

More details on the representation of fixed-point and floating-point numbers can be found, for example, in references [KUO-05] or [RAB-75].

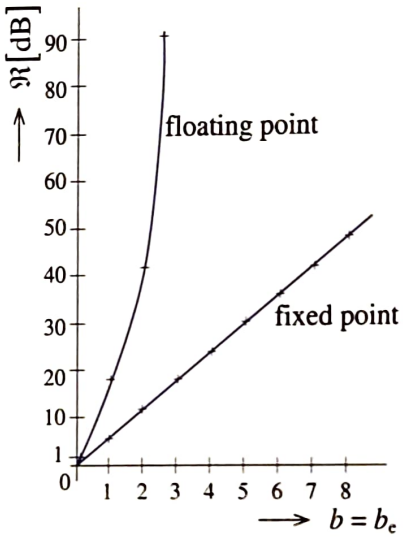


Figure 5.3: Comparison of dynamic ranges of type INT and type FLP systems

Sampling can be divided into three groups (Fig. 5.4)
- ideal sampling
- first-order sampling
- second-order sampling

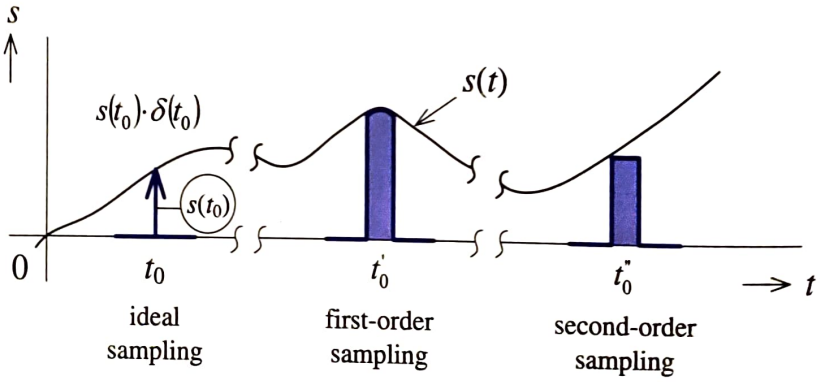


Figure 5.4: Types of sampling

If we want a continuous-time signal (continuous analog signal) to be processed by a discrete or digital system, we must use an A/D (analog-to-digital) and D/A (digital-to-analog) converter, as shown in Fig. 5.5.

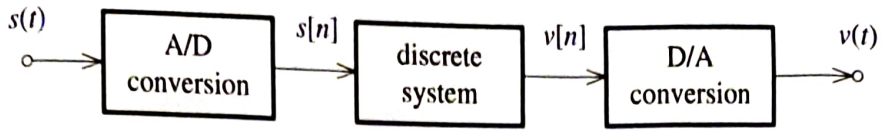


Figure 5.5: Processing of analog signal by discrete or digital system

5.2 Ideal sampling

A continuous-time signal $s(t)$ is transformed into a sequence of quantities represented by instantaneous values of the signal $s(t)$. This sequence is called the discrete signal $s[n]$ and the procedure is referred to as ideal sampling or discretization. The operation of the ideal sampling can be thought of as the product of the signal $s(t)$ and an infinite sequence of Dirac impulses, which are uniformly spaced one sampling interval T apart

$$s_{\mathrm{id}}(t) = s(t)s_{\delta}(t) = s(t)\sum_{n=-\infty}^{\infty}\delta(t-nT) = \sum_{n=-\infty}^{\infty}s[nT]\,\delta(t-nT) \tag{5.6}$$

To derive the Fourier transform $S_{\mathrm{id}}(\omega)$, which is the Fourier transform of $s_{\mathrm{id}}(t)$, we will make use of the following three results. We need the Fourier transform of the periodic sequence of Dirac impulses $s_{\delta}(t)$. To find it we first find the inverse Fourier transform of a shifted Dirac impulse in frequencies $\delta(\omega-\omega_s)$

$$\mathcal{F}^{-1}\{\delta(\omega-\omega_{\mathrm{s}})\} = \frac{1}{2\pi}\int_{-\infty}^{\infty}\delta(\omega-\omega_{\mathrm{s}})\mathrm{e}^{\mathrm{j}\omega t}\mathrm{d}\omega = \frac{1}{2\pi}\mathrm{e}^{\mathrm{j}\omega_{\mathrm{s}}t}$$

where we have used the sampling property of the Dirac impulse. So we have the pairs

$$\frac{1}{2\pi}\mathrm{e}^{\mathrm{j}\omega_{\mathrm{s}}t} \Leftrightarrow \delta(\omega-\omega_{\mathrm{s}}) \quad \mathrm{e}^{\mathrm{j}\omega_{\mathrm{s}}t} \Leftrightarrow 2\pi\delta(\omega-\omega_{\mathrm{s}}) \quad \mathrm{e}^{\mathrm{j}k\omega_{\mathrm{s}}t} \Leftrightarrow 2\pi\delta(\omega-k\omega_{\mathrm{s}})$$

Now we express the sequence $s_{\delta}(t)$ in the form of Fourier series. We use (2.20) where we have the c_k coefficients of the periodic sequence of rectangular impulses modified to our case

$$c_k = \frac{D\vartheta}{T}\cdot\frac{\sin\left(k\omega_{\mathrm{s}}\frac{\vartheta}{2}\right)}{k\omega_{\mathrm{s}}\frac{\vartheta}{2}}$$

Sequence $s_{\delta}(t)$ is the limit case for $\vartheta \to 0$ with $D\vartheta = 1$. The sinc function $\sin(x)/x$ at 0 is equal to 1, so we have constant coefficients $c_k = 1/T$. By inserting to (2.16) we get

$$s_{\delta}(t) = \sum_{n=-\infty}^{\infty}\delta(t-nT) = \sum_{k=-\infty}^{\infty}\frac{1}{T}\mathrm{e}^{\mathrm{j}k\omega_{\mathrm{s}}t}$$

Computing the Fourier transform of the right-hand side and using the above Fourier transform pair provides

$$\mathcal{F}\left\{\sum_{k=-\infty}^{\infty}\frac{1}{T}e^{jk\omega_s t}\right\}=\frac{1}{T}\sum_{k=-\infty}^{\infty}\mathcal{F}\{e^{jk\omega_s t}\}=\frac{1}{T}\sum_{k=-\infty}^{\infty}2\pi\delta(\omega-k\omega_s)$$

So we have the pair that shows the interesting fact that the spectrum of a periodic sequence of Dirac impulses in time is again a sequence of Dirac impulses in frequency domain.

$$\sum_{n=-\infty}^{\infty}\delta(t-nT)\Leftrightarrow\frac{2\pi}{T}\sum_{k=-\infty}^{\infty}\delta\left(\omega-k\frac{2\pi}{T}\right),\quad f_s=\frac{1}{T},\quad \omega_s=2\pi f_s \tag{5.7}$$

The second result is the correspondence between the product in the time domain and the convolution in the spectral domain

$$s_1(t)\cdot s_2(t)\Leftrightarrow\frac{1}{2\pi}\cdot S_1(\omega)*S_2(\omega)=\frac{1}{2\pi}\int_{-\infty}^{\infty}S_1(\xi)\,S_2(\omega-\xi)\,d\xi \tag{5.8}$$

The third result is the sampling property of the Dirac impulse that also holds in the spectral domain

$$S(\omega)*\delta(\omega-\Omega)=\int_{-\infty}^{\infty}S(\xi)\,\delta(\omega-\Omega-\xi)\,d\xi=S(\omega-\Omega) \tag{5.9}$$

When deriving the Fourier transform $S_{id}(\omega)$ we will successively use all these three properties

$$\begin{aligned}
S_{id}(\omega)&=\mathcal{F}\{s_{id}(t)\}=\mathcal{F}\left\{s(t)\sum_{n=-\infty}^{\infty}\delta(t-nT)\right\}=\frac{1}{2\pi}\cdot S(\omega)*\mathcal{F}\left\{\sum_{n=-\infty}^{\infty}\delta(t-nT)\right\}\\
&=\frac{1}{2\pi}S(\omega)*\frac{2\pi}{T}\sum_{k=-\infty}^{\infty}\delta\left(\omega-k\frac{2\pi}{T}\right)=S(\omega)*\frac{1}{T}\sum_{k=-\infty}^{\infty}\delta(\omega-k\omega_s)\\
&=\frac{1}{T}\,S(\omega)*\{\cdots+\delta(\omega+2\omega_s)+\delta(\omega+\omega_s)+\delta(\omega)+\delta(\omega-\omega_s)+\delta(\omega-2\omega_s)\cdots\}\\
&=\frac{1}{T}\,\{\cdots+S(\omega+2\omega_s)+S(\omega+\omega_s)+S(\omega)+S(\omega-\omega_s)+S(\omega-2\omega_s)\cdots\}
\end{aligned}$$

$$S_{id}(\omega)=\mathcal{F}\{s_{id}(t)\}=\frac{1}{T}\sum_{k=-\infty}^{\infty}S(\omega-k\omega_s),\quad \text{when}\quad S(\omega)=\mathcal{F}\{s(t)\} \tag{5.10}$$

The Fourier transform of the ideally sampled signal $s_{id}(t)$ with a period T is equal to a superposition of uniformly shifted spectra of the original signal, multiplied by the constant $1/T$. Fig. 5.6 gives the wavelengths and spectra of the continuous signal $s(t)$ and the ideally sampled signal $s_{id}(t)$. The spectrum of a discrete signal is always periodic, with the period equal to the sampling frequency!

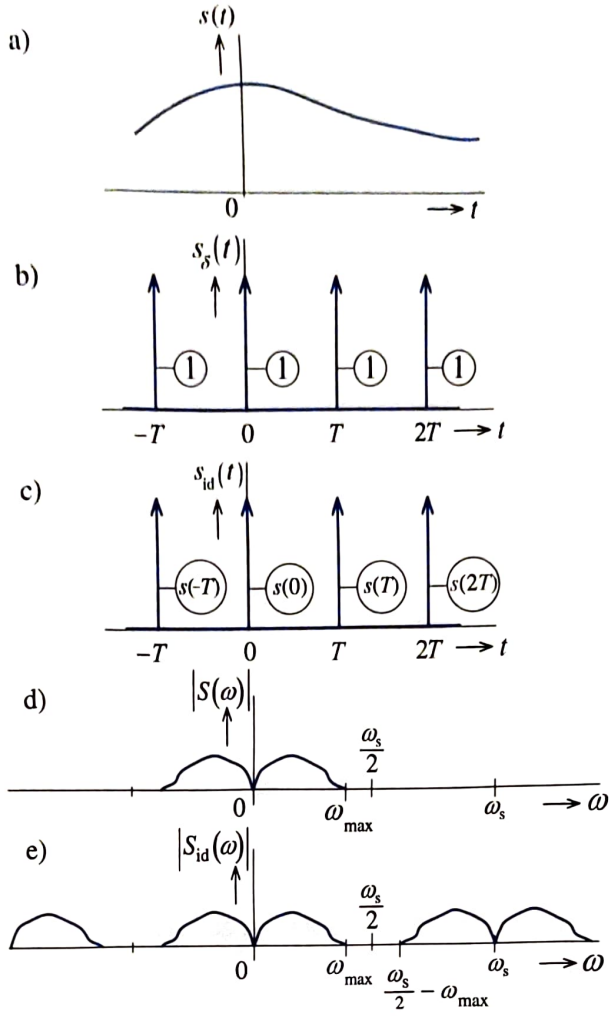


Figure 5.6: Ideal sampling: a) continuous-time signal $s(t)$, b) sequence of Dirac impulses $s_\delta(t)$, c) ideally sampled signal $s_{id}(t)$, d) module of spectrum of continuous signal $S(\omega)$, e) module of spectrum of ideally sampled signal $S_{id}(\omega)$

For the spectra of an ideally sampled signal not to overlap, the following inequality must hold

$$\frac{\pi}{T} > \omega_{max} \quad \text{or} \quad f_s > 2f_{max} \tag{5.11}$$

where f_{max} is the maximum frequency of a component of the spectrum $S(\omega)$, which still has a non-zero value. Relation (5.11) is called the **sampling theorem** (Nyquist theorem, Shannon-Kotelnikov theorem, etc.). If the sampling theorem does not hold, neighbouring spectra will overlap and it is no longer possible to recover without errors the original Fourier transform $S(\omega)$, and thus the original waveform of the signal $s(t)$ cannot be recovered either. This situation, namely when the sampling theorem does not hold, is shown in Fig. 5.7.

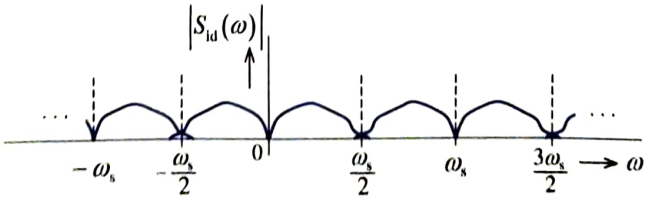


Figure 5.7: Fourier transform of sampled signal, with condition (5.11) not satisfied